Qualcomm Technologies, Inc.

AudioReach Signal Processing Framework Software Porting Manual for Linux Android

80-VN500-14 Rev. AC

November 23, 2021

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:39 GMT nijiasheng@longcheer.com

For additional information or to submit technical questions, go to: https://createpoint.qti.qualcomm.com

Confidential – Qualcomm Technologies, Inc. and/or its affiliated companies – May Contain Trade Secrets

NO PUBLIC DISCLOSURE PERMITTED: Please report postings of this document on public servers or websites to: DocCtrlAgent@qualcomm.com.

Confidential Distribution: Use or distribution of this item, in whole or in part, is prohibited except as expressly permitted by written agreement(s) and/or terms with Qualcomm Incorporated and/or its subsidiaries.

Not to be used, copied, reproduced, or modified in whole or in part, nor its contents revealed in any manner to others without the express written permission of Qualcomm Technologies, Inc.

All Qualcomm products mentioned herein are products of Qualcomm Technologies, Inc. and/or its subsidiaries.

Qualcomm is a trademark or registered trademark of Qualcomm Incorporated. Other product and brand names may be trademarks or registered trademarks of their respective owners.

This technical data may be subject to U.S. and international export, re-export, or transfer (“export”) laws. Diversion contrary to U.S. and international law is strictly prohibited.

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
U.S.A.

© 2020-2021 Qualcomm Technologies, Inc. and/or its subsidiaries. All rights reserved.

Revision history

Revision	Date	Description
AA	July 2020	Initial release
AB	December 2020	Updated Chapter 2
AC	November 2021	Editorial updates. No technical content was changed.

Qualcomm
Confidential - May Contain Trade Secrets
2022-08-10 09:52:39 GMT
nijiasheng@longcheer.com

Contents

1 Introduction 5

1.1 Purpose 5
1.2 Conventions 5
1.3 Technical assistance 5

2 HLOS audio software integration 6

2.1 High-level software architecture 6
2.2 HLOS audio kernel drivers 7
2.3 Device tree 9
2.4 HLOS user space modules 10
2.4.1 Audio HAL 11
2.4.2 SoundTrigger HAL 11
2.4.3 PAL 11
2.4.4 Sound card parser 12
2.4.5 TinyALSA plugin 12
2.4.6 Audio graph manager (AGM) 12
2.4.7 Graph service layer (GSL) 13
2.4.8 ACDB management layer (AML) 13
2.4.9 Operating system abstraction layer (OSAL) 13
2.4.10 AudioReach utilities 13
2.4.11 AudioReach tuning service 14
2.4.12 SPF 14
2.4.13 Generic packet router (GPR) 14
2.4.14 ACDB data 14
2.4.15 Audio factory test model (FTM) 15

3 SPF 16

3.1 High-level SPF architecture 16
3.2 Top-level folder structure 17
3.2.1 Build folder structure 18
3.2.2 FWK SPF folder structure 19
3.2.3 FWK platform folder structure 20
3.2.4 FWK API folder structure 21
3.2.5 FWK pack folder structure 21
3.2.6 FWK build folder structure 21
3.2.7 SPM/CAPI module folder structure 22

4 MPSS audio software integration 23

4.1 MPSS audio architecture 23
4.2 AVS folder structure 24

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

4.2.1 API folder structure 24
4.2.2 VSD folder structure....25
4.3 GPR folder structure 26
4.4 Build level updates....26

A References.... 27
A.1 Acronyms and terms....27

Qualcomm
Confidential - May Contain Trade Secrets
2022-08-10 09:52:39 GMT
nijiasheng@longcheer.com

1 Introduction

1.1 Purpose

The document describes software integration for audio kernel components, user space audio modules, and APIs in the AudioReach™ architecture's signal processing framework (SPF).

1.2 Conventions

Function declarations, function names, type declarations, attributes, and code samples appear in a different font, for example, `#include`.

Code variables appear in angle brackets, for example, `<number>`.

Commands to be entered appear in a different font, for example, **`copy a:*.* b:`**.

Button and key names appear in bold font, for example, click **Save** or press **Enter**.

Shading indicates content that has been added or changed in this revision of the document.

1.3 Technical assistance

For assistance or clarification on information in this document, submit a case to Qualcomm Technologies, Inc. (QTI) at https://createpoint.qti.qualcomm.com/.

If you do not have access to the CDMATech Support website, register for access or send email to support.cdmatech@qti.qualcomm.com.

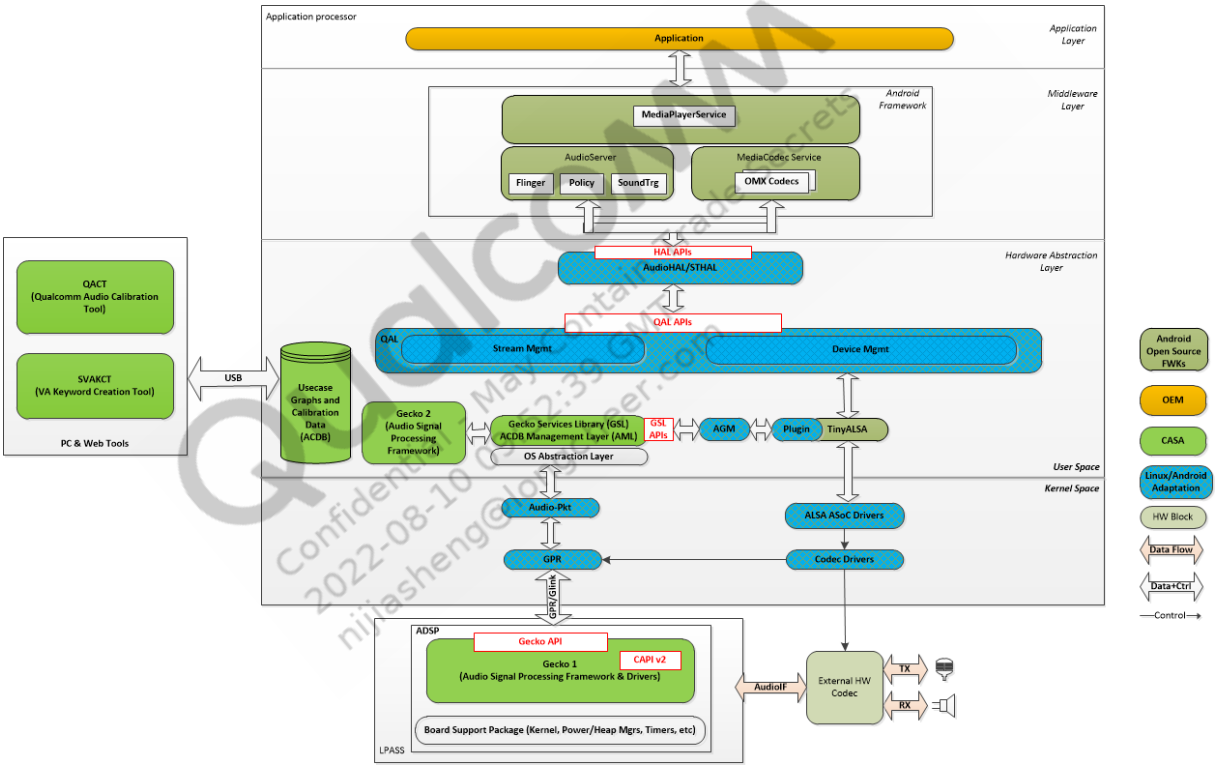
Qualcomm Confidential – May Contain Trade Secrets 2022-08-10 09:39:35 GMT xiaoheng@longchuer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2 HLOS audio software integration

2.1 High-level software architecture

The following diagram shows the high-level software architecture.



Qualcomm Confidential – May Contain Trade Secrets 2022-08-10 08:32:39 GMT niijasheng@lo.com

2.2 HLOS audio kernel drivers

- Audio driver supports kernel 5.4
- Git project – vendor/qcom/opensource/audio-kernel.git
- The same git project and same folder structure used in the Elite architecture are also used in the new AudioReach architecture
- IPC driver
  - Audio packet driver – Exposes char device to the user space to receive and send GPR packets from the user space GSL
    - File – vendor/qcom/opensource/audio-kernel/ipc/audio-pkt.c
  - GPR driver – Sends and receives GPR packets through RPMSG/Glink to ADSP
    - File – vendor/qcom/opensource/audio-kernel/ipc/gpr-lite.c
- ALSA ASOC drivers
  - Machine driver – Target-specific configuration. New AudioReach architecture removes all front-end DAI links and CPU DAI links.
    - File – vendor/qcom/opensource/audio-kernel/asoc/lahaina.c
  - Common utility driver – Channel map mixer commands used by AGM, audio hardware interfaces such as MI2S/TDM GPIO pinctrl states, and sysfs node to receive audio service status
    - File – vendor/qcom/opensource/audio-kernel/ipc/msm-common.c
- ALSA codec drivers
  - No changes in these drivers. The same drivers are used in the AudioReach architecture.
    - Folder – vendor/qcom/opensource/audio-kernel/asoc/codecs
- DSP drivers
  - Audio-prm driver – Used for codec and hardware interface clock control
    - File – vendor/qcom/opensource/audio-kernel/dsp/audio_prm.c
  - Spf-core-platform driver – This driver is equivalent to the q6-core driver in the Elite architecture. It adds all child devices (machine, codec devices, etc.) and checks for APM ready from the ADSP.
    - File – vendor/qcom/opensource/audio-kernel/dsp/spf-core.c
  - ADSP loader driver – Same driver present in the Elite architecture
    - File – vendor/qcom/opensource/audio-kernel/dsp/adsp-loader.c
  - ION memory driver – Same driver present in the Elite architecture. Allocations happen in the user space
    - File – vendor/qcom/opensource/audio-kernel/dsp/msm-audio-ion.c

Qualcomm Confidential May Contain Trade Secrets 2022-03-10 09:52:39 GMT nyusheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

- Audio SSR driver – Same driver present in the Elite architecture
  - File – vendor/qcom/opensource/audio-kernel/dsp/audio_ssr.c
- Audio PD driver – Same driver present in the Elite architecture
  - File – vendor/qcom/opensource/audio-kernel/dsp/audio_pdr.c
- Audio notifier driver – Same driver present in the Elite architecture
  - File – vendor/qcom/opensource/audio-kernel/dsp/audio_notifier.c
- Audio event notifier driver – Same driver present in the Elite architecture. Sends events from the ADSP to the codec drivers regarding headset insertion/removal and speaker protection.
  - File – vendor/qcom/opensource/audio-kernel/dsp/msm-audio-event-notify.c
- Voice MHI driver – Same driver present in the Elite architecture. Not needed for Audio kernel 5.4 and later.
  - File – vendor/qcom/opensource/audio-kernel/dsp/voice_mhi.c
- Digital codec clock driver – Same driver present in the Elite architecture
  - File – vendor/qcom/opensource/audio-kernel/dsp/digital-cdc-rsc-mgr.c
- Speaker protection parameters driver – Same driver present in the Elite architecture
  - File – vendor/qcom/opensource/audio-kernel/dsp/sp_params.c

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:57 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.3 Device tree

- Audio device tree supports kernel 5.4
- Git project – kernel/devicetree.git
- The same git project, folder structure, and files used in the Elite architecture are also used in the new AudioReach architecture
- All platform and CPU-DAI device tree nodes are no longer needed and are removed in AudioReach
- The following are the new device tree nodes created for the AudioReach architecture:
  - File – vendor/qcom/proprietary/devicetree-5.4/qcom/lahaina-audio.dtsi)

```
&spf_core_platform {
       ….
       ….
       lahaina_snd: sound {
               compatible = "qcom, lahaina-asoc-snd"
                ….
       }
    /* other Codec Clock Vote DT Nodes */
 }
```

  - File – vendor/qcom/proprietary/devicetree-5.4/qcom/lahaina.dtsi

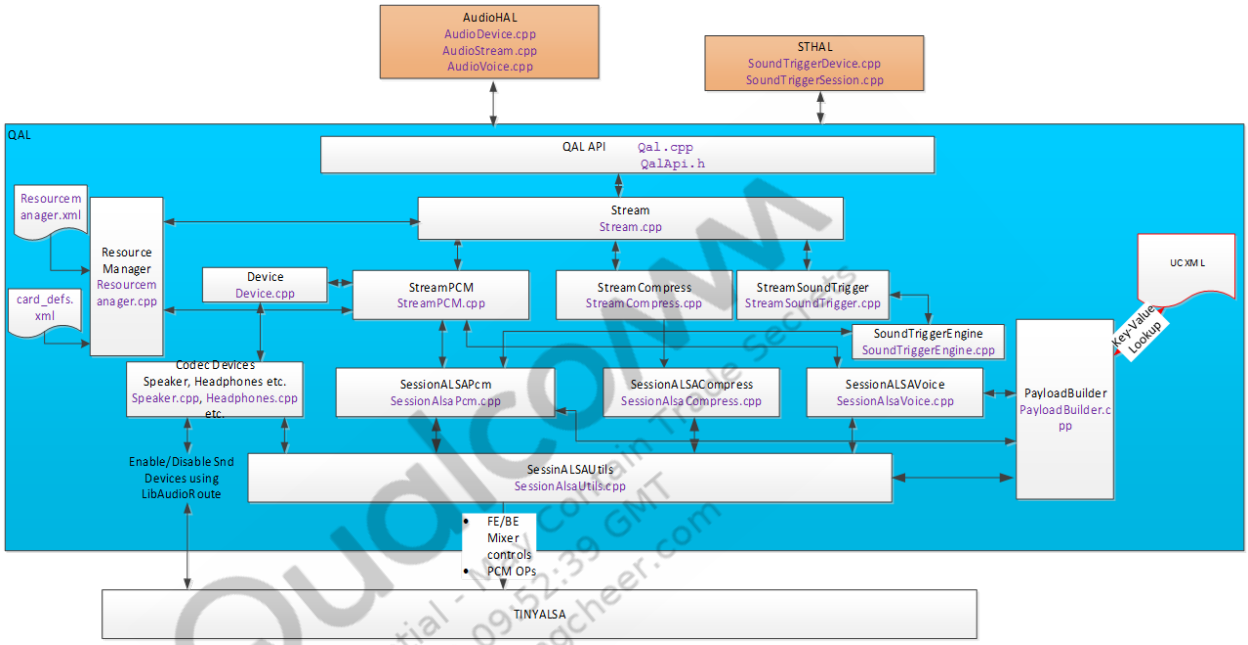
```
  Audio_gpr: qcom,gpr {
    Compatible = "qcom,gpr"
    ….
    }
    Spf_core {
       Compatible = "qcom,spf_core"
     ….
      }
       audio-packet {
          compatible ="qcom,audio-pkt"
          ….
          }
    …..
   }
```

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:39 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.4 HLOS user space modules

- Audio user space modules support Android R
- The following image shows the audio HAL, SoundTrigger HAL, and Qualcomm audio layer (QAL) folder structure



QAL's name has been changed to PAL. Qal.cpp is changed to pal.cpp, QalApi.h to palapi.h.

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:39 GMT nijiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.4.1 Audio HAL

Git project – vendor/qcom/opensource/audio-hal/audio-android.git

- Android's audio HAL connects the higher-level, audio-specific framework APIs in android.media to the underlying audio driver and hardware. The audio HAL can directly access the QAL.
- Implemented in C++
- Source code folder – /vendor/qcom/opensource/audio-hal/primary-hal
- Implemented library – audio.primary.<target name>.so
- When compiling the AudioReach Code, the make file in primary-hal should have the following files included

```
include $(MY_LOCAL_PATH)/hal-pal/Android.mk
     include $(MY_LOCAL_PATH)/hal-pal/audio_extn/Android.mk
```

2.4.2 SoundTrigger HAL

- Git project – vendor/qcom/opensource/audio-hal/st-hal.git
- The SoundTrigger HAL connects the higher-level, audio-specific SoundTrigger framework APIs to the underlying audio driver and hardware. The SoundTrigger HAL can directly access the PAL.
- Implemented in C++
- Source code folder – /vendor/qcom/opensource/audio-hal/st-hal/sthal-pal
- Implemented library – sound_trigger.primary.<target name>.so
- When compiling the AudioReach Code, the make file in st-hal should have the following files included

```
include $(LOCAL_PATH)/sthal-pal/Android.mk
```

2.4.3 PAL

- Git project – vendor/qcom/opensource/pal.git
- A new git project was created for the PAL. The PAL provides high-level audio-specific APIs to access the underlying audio hardware and drivers.
- Implemented in C++
- Source code folder – /vendor/qcom/opensource/pal
- Implemented library – libpal.so

Qualcomm Confidential - May Contain Trade Secrets 2022-08-30 09:52:35 PDT hijacheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.4.4 Sound card parser

- Git project – vendor/qcom/opensource/agm.git
- Defines virtual cards and devices. Hardware card/device description is not required. If (card, device) is not defined in card-defs, then it is assumed to be a hardware card and device.
- Implemented in C
- Source code folder – /vendor/qcom/opensource/agm/snd_parser
- Implemented library – libsndcardparser.so
- Dependency library – libexpat.so, libcutils.so

2.4.5 TinyALSA plugin

- Git project – vendor/qcom/opensource/agm.git
- PCM, mixer, and compress TinyALSA plugins route all PCM, mixer, and compress calls from the application to plugin-specific implementation
- Implemented in C
- Source code folder – /vendor/qcom/opensource/agm/plugins/tinyalsa
- Implemented library – libagm_pcm_plugin.so, libagm_compress_plugin.so, libagm_mixer_plugin.so
- Dependency library – libtinyalsa.so, libsndparser.so, libagmclient.so, libutils.so, libcutils.so, liblog.so

2.4.6 Audio graph manager (AGM)

- Git project – vendor/qcom/opensource/agm.git
- A new git project was created for AGM. AGM provides APIs for mixer plugins, PCM APIs, and compressed APIs to set up audio use cases. It runs as an audio service in a separate process.
- Implemented in C
- Source code folder – /vendor/qcom/opensource/agm
- Implemented library – libagm.so
- Dependency library – libar-gsl.so, liblog.so, liblx-osal, libaudioroute.so, libats.so, libtinyalsa.so

Qualcomm Confidential - May Contain Trade Secrets 2023-08-10 03:52:39 GMT mjiasheng@longsheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.4.7 Graph service layer (GSL)

- Git project – vendor/qcom/proprietary/args.git
- GSL is the SPF service layer responsible for loading and initializing graphs and subgraphs via graph key vector. It also handles graph setup, dynamic handling of subgraphs within the graph, and data command management including read/write buffer and calibration (set config/get config) of SPF modules.
- Implemented in C
- Source code folder – /vendor/qcom/proprietary/args/gsl
- Implemented library – libar-gsl.so
- Dependency library – liblx-osal.so, libar-acdb.so, liblx-ar_util.so, libar-gpr

2.4.8 ACDB management layer (AML)

- Git project – vendor/qcom/proprietary/args.git
- AML provides get/set APIs to retrieve and adjust data in the ACDB DATA files. It provides data abstractions and organization for how calibration data is to be consumed by the audio driver and its components.
- Implemented in C
- Source code folder – /vendor/qcom/proprietary/args/acdb
- Implemented library – libar-acdb.so
- Dependency library – liblx-osal.so

2.4.9 Operating system abstraction layer (OSAL)

- Git project – vendor/qcom/proprietary/args.git
- OS abstraction layer implementation
- Implemented in C
- Source code folder – /vendor/qcom/proprietary/args/ar-osal
- Implemented library – liblx-osal.so

2.4.10 AudioReach utilities

- Git project – vendor/qcom/proprietary/args.git
- AudioReach utilities implementation (for example, linked list, etc.)
- Implemented in C
- Source code folder – /vendor/qcom/proprietary/args/ar-util
- Implemented library – liblx-ar_util.so
- Dependency library – liblog.so, liblx-osal.so

Qualcomm Confidential – May Contain Trade Secrets 2026-08-10 09:52:53 GMT mjiasheng@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

2.4.11 AudioReach tuning service

- Git project – vendor/qcom/proprietary/args.git
- AudioReach tuning services such as RTC, MCS, FTS, and RTGM
- Implemented in C
- Source code folder – /vendor/qcom/proprietary/args/ats
- Implemented library – libats.so
- Dependency library – liblx-osal.so, libutils.so, libcutils.so, libdiag.so, libar-gsl.so, libar-acdb.so

2.4.12 SPF

- Git project – vendor/qcom/proprietary/args.git
- SPF files used by AGM and QAL
- Implemented in C
- Source code folder – /vendor/qcom/proprietary/args/spf

2.4.13 Generic packet router (GPR)

- Git project – vendor/qcom/proprietary/args.git
- GSL uses GPR protocol to communicate messages with the SPF on the ADSP. Since SPF runs on a different subsystem, GSL has to communicate over an IPC layer. This IPC is implemented as a device node which is opened by the user space GSL. The IPC mechanism depends on the Linux kernel version. Based on the kernel version, this driver maps and interacts with the corresponding kernel IPC layer (for example, Glink on kernel 4.09, RPMSG in kernel 4.14 and onward).
- Implemented in C
- Source code folder – /vendor/qcom/proprietary/args/gpr
- Implemented library – libar-gpr.so
- Dependency library – liblx-osal.so, libcutils.so, liblog.so

2.4.14 ACDB data

- Git project – vendor/qcom/proprietary/mm-audio.git
- All ACDB data graphs and kvh2xml headers are placed in this folder
- Source code folder – /vendor/qcom/proprietary/mm-audio/ar-acdb

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 05:52:29 GMT nijesh.kalarickal@cheer.com

2.4.15 Audio factory test model (FTM)

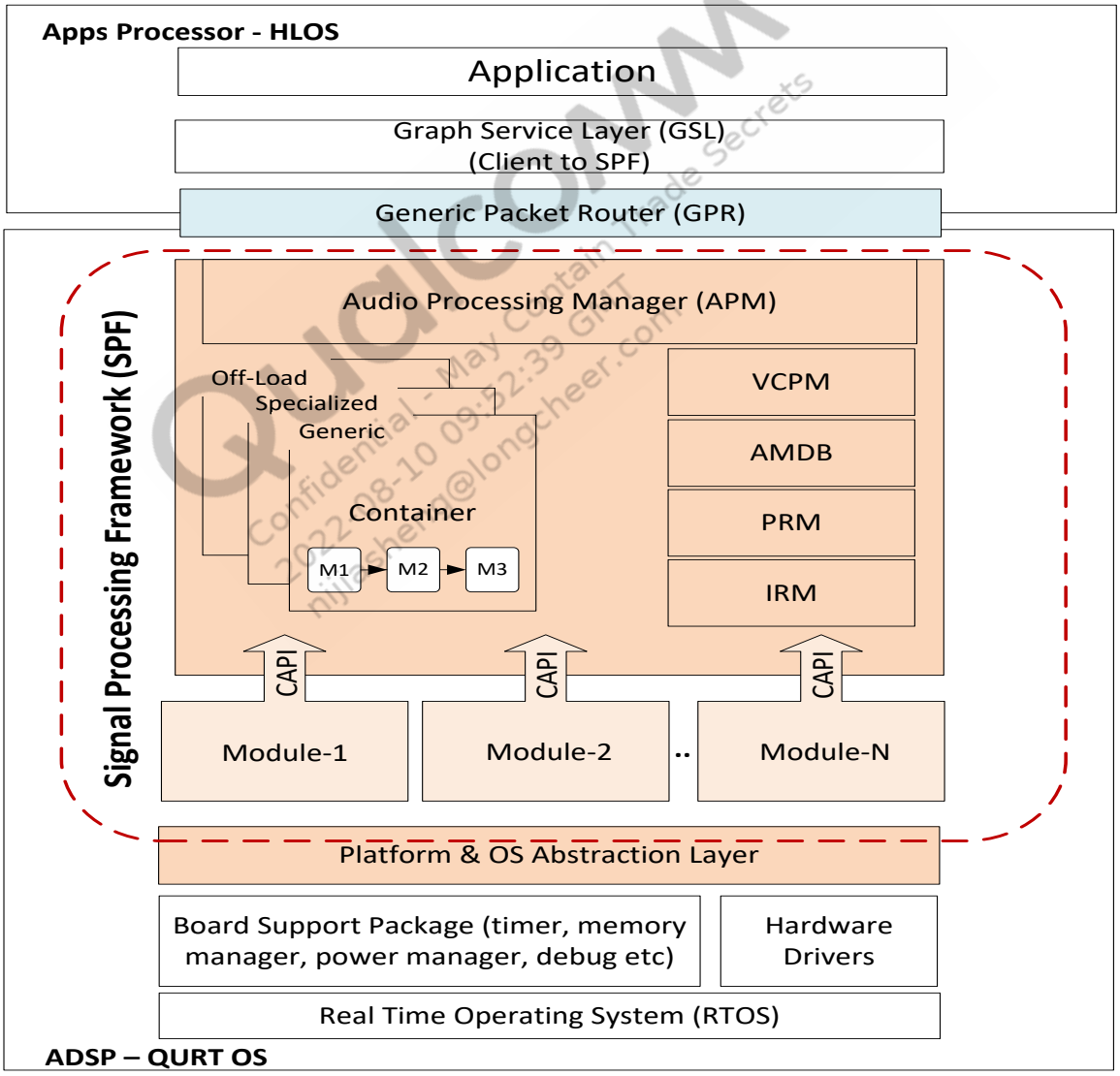
- Git project – vendor/qcom/proprietary/mm-audio.git
- Audio FTM code
- Source code folder – /vendor/qcom/proprietary/mm-audio/audio_ftm/casa
- Implemented executable – mm-audio-ftm

Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:39 GMT nijiasheng@longcheer.com

3 SPF

3.1 High-level SPF architecture

The following diagram shows the high-level SPF architecture.

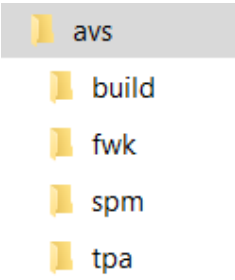


Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:22:39 GMT nijusheer@longcheer.com

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.2 Top-level folder structure

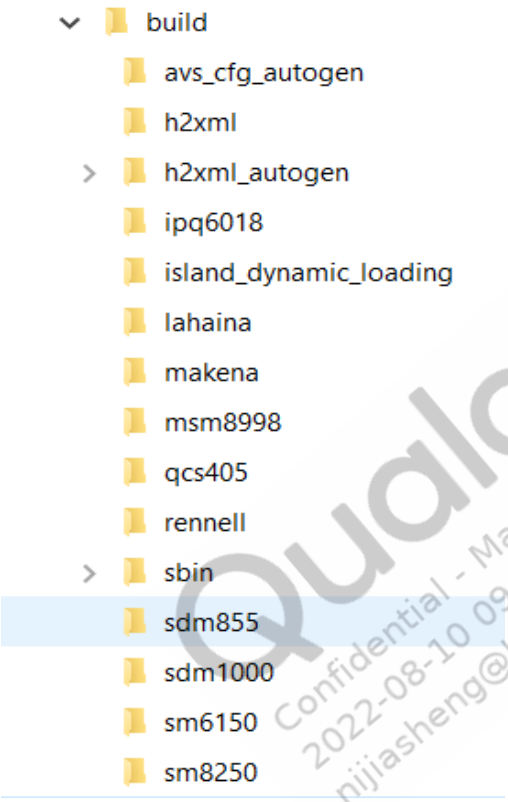
- Build – Per target-based build configurations
- FWK – Framework public APIs.
  - APM, containers, control modules, POSAL
  - Framework modules
  - Framework utilities
  - Hardware drivers
- SPM – Signal processing CAPI-wrapped libraries
- TPA – Third party algorithms (for example, WMA decoder)



Qualcomm
Confidential - May Contain Trade Secrets
2022-08-10 09:52:39 GMT
nijiasheng@longcheer.com

3.2.1 Build folder structure

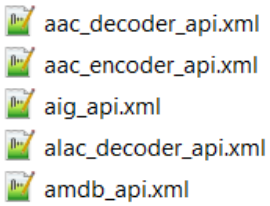
- Per target-based list of active and stubbed libraries
- HAL version configuration for different hardware drivers (for example, DMA, I2S, PCM/TDM, etc.)
- H2XML generation utilities



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:39 GMT nijiasheng@longcheer.com

h2xml_autogen

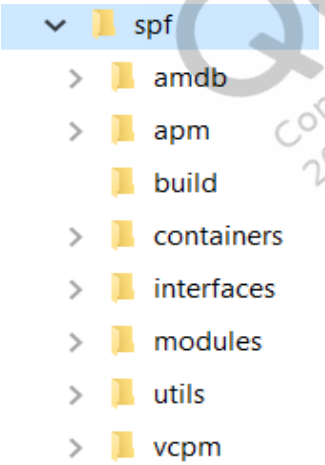
Contains autogenerated XML files for all the module's public APIs in the framework. The following image shows an example.



MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

3.2.2 FWK SPF folder structure

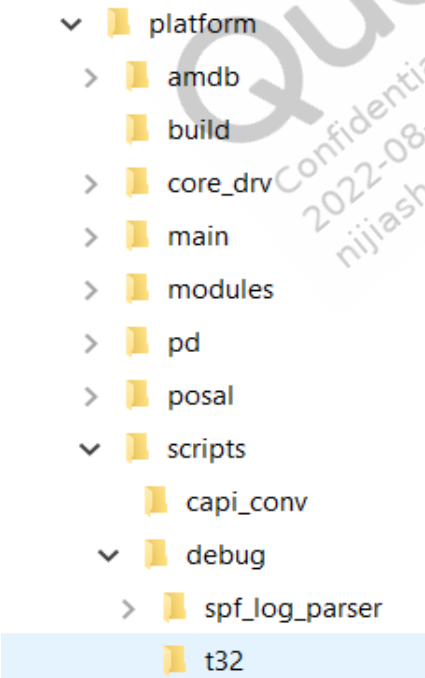
- APM, containers, VCPM, AMDB
- Interfaces
  - FWK internal APIs
    - APM – Container/control modules
    - Container – Container
    - Container – Control modules
    - Offload related
    - Container – VCPM
  - CAPI-related APIs
    - IMCL, met data, CAPI common utils, etc.
  - MDF-related internal APIs
  - Modules – Framework modules (for example, data logging, accumulator, splitter, sync, etc.)
- Utils
  - SPF messaging framework
  - Common utility functions (for example, linked list, circular buffer, etc.)



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:19 GMT nijiasheng@longcheer.com

3.2.3 FWK platform folder structure

- POSAL, per target device configuration
- Hardware and shared memory related end point modules
- Hardware accelerators (for example, hardware resampler, HW-AF, LPMLA, etc.)
- PRM, IRM
- Scripts
  - CAPI conversion script
    - avs\fwk\platform\scripts\capi_conv\elite_to_spf.py
  - SPF log parser
    - avs\fwk\platform\scripts\debug\spf_log_parser
    - Contains graph rendering scripts for diag logs captured from various sources including QXDM, crashman, crashscope, OTTP, and simulator
  - T32
    - avs\fwk\platform\scripts\debug\t32\graph_info.cmm
    - T32 cmm script to display the module graph (for example, after loading RAM dumps on T32)



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:39 GMT nijiasheng@longcheer.com

3.2.4 FWK API folder structure

Contains public APIs for:

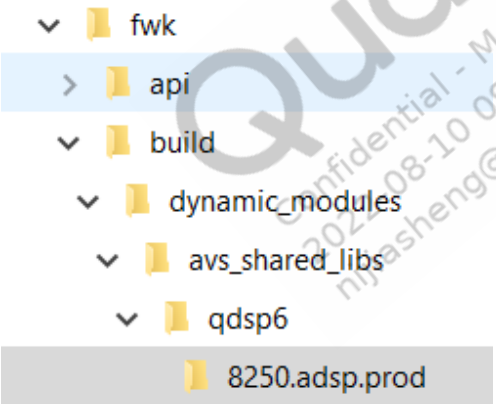
- APM, control modules (for example, VCPM, IRM, PRM, and AMDB)
- Framework modules (for example, data logging, accumulator, splitter, etc.)
- Hardware endpoints (for example, I2S, PCM, TDM, etc.)
- Shared memory endpoints (for example shared memory read/write, push-pull modules, mailbox, etc.)
- Module common APIs, common media formats, and per encoder/decoder media formats

3.2.5 FWK pack folder structure

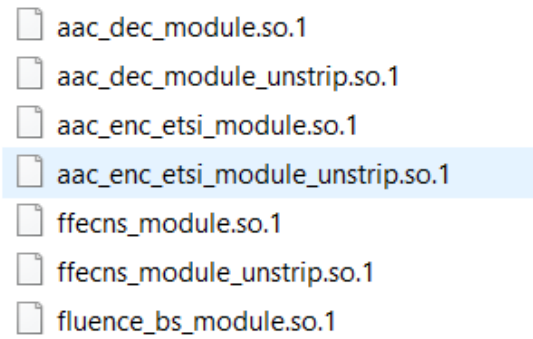
Contains scripts for HY11 and open source-related package creations.

3.2.6 FWK build folder structure

Contains compile time generated shared objects for dynamically loadable CAPI wrapped modules.



Shared objects:

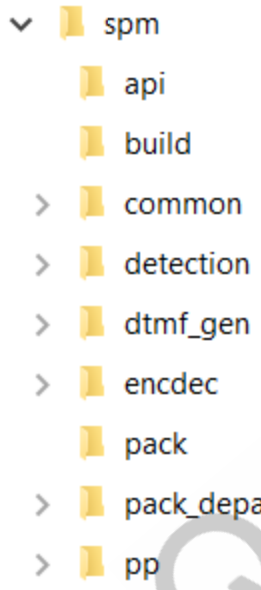


Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:39 GMT nlyasheng@longcheer.com

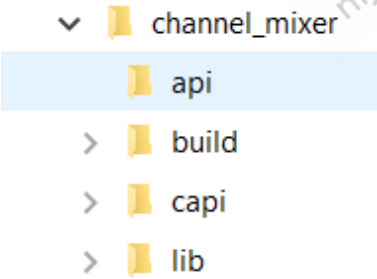
3.2.7 SPM/CAPI module folder structure

- Contains CAPI-wrapped signal processing libraries categorized as per their functionalities (for example, encoder, decoder, pre- and postprocessing, etc.)
- Each CAPI-wrapped module has API, CAPI, and lib folders
- API – Contains control port intent ID definitions

SPM folder structure:



CAPI module folder structure:

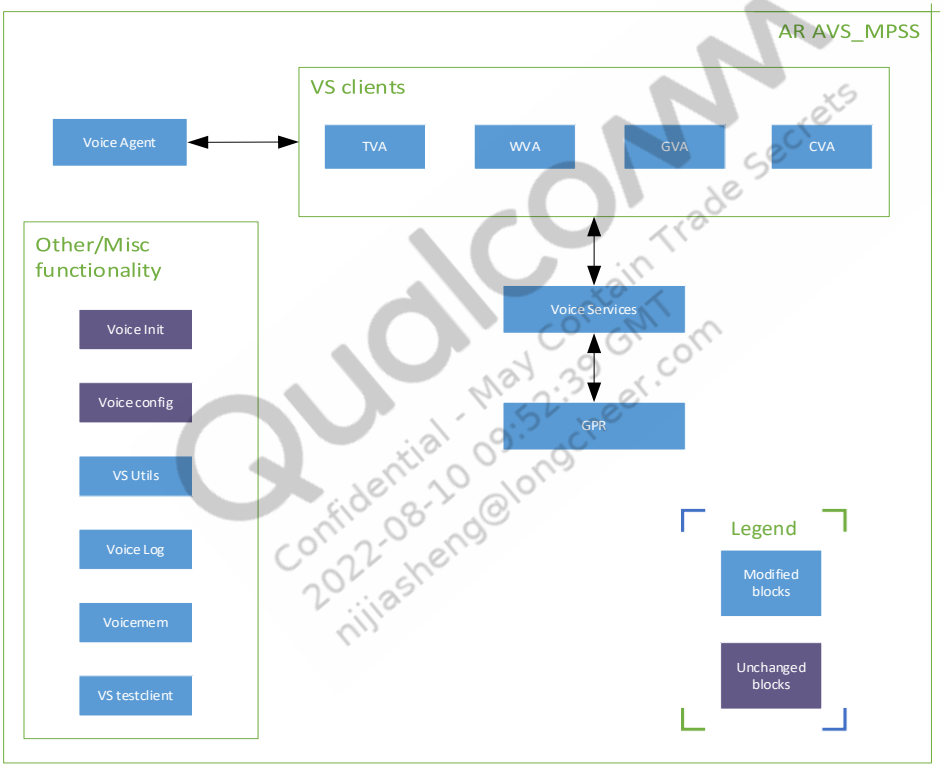


Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:39 GMT nijiasheng@longcheer.com

4 MPSS audio software integration

4.1 MPSS audio architecture

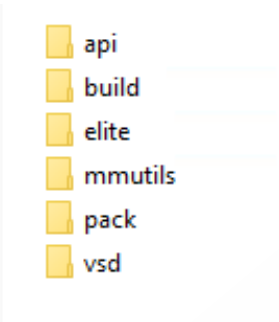
The following diagram shows the MPSS audio architecture.



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:39 GMT nijiasheng@longcheer.com

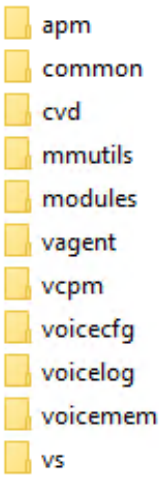
4.2 AVS folder structure

- API – Contains newly introduced SPF APIs
- Elite (new folder)
  - Contains the Elite architecture codebase. This is maintained to facilitate mapping of common codebase between the old and new architecture.
  - Will not be compiled for AR-specific builds. This decision is taken at the build level based on the USES_AUDIO_GPR flag.
- Pack and build – Corresponding changes for newly added files
- MMutils – No change
- VSD – Driver level changes



4.2.1 API folder structure

The APM, VCPM, and modules folders are newly introduced and contain the respective SPF APIs.



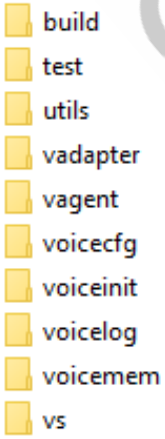
Qualcomm Confidential - May Contain Trade Secrets 2022-03-10 09:52:39 GMT nijiasheng@longcheer.com

4.2.2 VSD folder structure

- The overall folder structure has not changed
- AR-specific changes are spread across the following folders:
  - Test
  - Utils
  - Vadapter
  - Vagent
  - Voicelog
  - Voicemem
  - VS

Summary of changes:

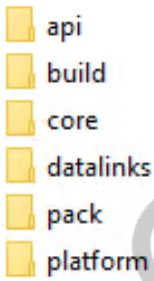
- Moved to AR OSAL APIs for thread creation/maintenance functionality
- Moved to GPR APIs for performing IPC operations between MPSS and ADSP
- Moved to SPF APIs for communication with ADSP
- VS utils upgraded to cover timer, object, and memory management functionalities
- There is no change with regards to APIs exposed to the respective RAT-specific protocol layers. Hence, there is no impact to non-audio layers in the modem.



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:06 GMT nijiasheng@longcheer.com

4.3 GPR folder structure

- In the Elite architecture, APR was used for communication between MPSS and ADSP during the voice call use case. In the AudioReach architecture, GPR is used.
- API – GPR APIs exposed to the client to send/receive packets
- Pack and build – Corresponding changes for newly added files
- Core
  - Core API, memory queue implementation
  - Local routing – Routing between two services on the same domain
  - Data links
  - Remote routing – Routing between two services on different domains
  - Platform – Platform specific configuration wrappers
  - AR OSAL implementation



Qualcomm Confidential - May Contain Trade Secrets 2022-08-10 09:52:39 GMT nijasheng@longcheer.com

4.4 Build level updates

The following files were updated:

- modem_proc/scripts/bsp/audio_root_libs/build/audio_root_libs.scons
- modem_proc/config/default/target_variation.xml

MAY CONTAIN U.S. AND INTERNATIONAL EXPORT CONTROLLED INFORMATION

A References

A.1 Acronyms and terms

Acronym or term	Definition
ACDB	Audio Calibration Data Base
AGM	Audio Graph Manager
ALSA	Advanced Linux Sound Architecture
AML	ACDB Management Layer
APM	Audio Processing Manager
CAPI	Common Audio Processing Interface
FTM	Factory Test Mode
GPR	Generic Packet Router
GSL	Graph Service Layer
IPC	Inter Process Communication
OSAL	Operating System Abstraction Layer
QAL	Qualcomm Audio Layer
RAT	Radio Access Technology
SPF	Signal Processing Framework
VCPM	Voice Call Proxy Manager
VSD	Voice Services and Drivers

Qualcomm Confidential – May Contain Trade Secrets
2022-03-04 23:59 GMT
hyacinthe.dengcher.com